HPML Final Project · Spring 2026

Performance Optimization of the TSFM Agent in an Industrial Agentic Benchmark

Optimization of the TSFM MCP agent in IBM's AssetOpsBench industrial AI benchmark, achieving a **3.3x reduction in workflow completion latency**

Team 27
Alisha Vinod · av3311
Thomas Ajai · tl3444
Jonathan Ang · ma4624
Sanjaii Vijayakumar · sv2851

GitHub: https://github.com/alishavinod/AssetOpsBench/tree/baseline
Tracking: https://wandb.ai/av3311-columbia-university/hpml-tsfm-optimization/workspace?nw=nwuserav3311

Dr. Kaoutar El Maghraoui · Columbia University · Week of May 4, 2026

Motivation & Problem

Why this matters

- Industrial maintenance workflows depend on fast, reliable TSFM forecasting.
- Current MCP tool calls add latency from model loading, data filtering, and inference.
- A naive baseline makes workflows slower and harder to scale across many sensors.

Problem Statement & Goal

We profile and optimize the TSFM MCP agent in IBM's AssetOpsBench benchmark, targeting latency bottlenecks across data quality filtering, model loading, and forecasting inference.

Goal:

Reduce per-tool-call and end-to-end workflow latency through five targeted optimizations while preserving forecast quality.

Technical Approach — Model & Data

Model

TinyTimeMixer (TTM) - IBM Research

- Parameter count: ~1M (ttm_96_28), ~5M (ttm_512_96)
- Framework: HuggingFace Transformers + granite-tsfm library
- Two variants: ttm_96_28 (context length 96, horizon 28) and ttm_512_96 (context length 512, horizon 96)

Dataset

IBM AssetOpsBench - Chiller 6 Sensor Data

- chiller6_june2020_sensordata_couchdb
- 2,896 rows × 12 sensor columns
- License: Apache 2.0 (IBM AssetOpsBench)
- No train/test splits - zero-shot inference only
- Sensors: temperature, efficiency, tonnage, power input, flow rate, refrigerant pressure

Hardware

- GPU: NVIDIA T4 (GCP n1-standard-4 instance)
- VRAM: 16GB
- CUDA: 12.4
- PyTorch: 2.6
- Orchestrator LLM: IBM Granite 3.3 8B Instruct via watsonx.ai API

Technical Approach — Optimizations Applied

1. Model Pre-loading and torch.compile

The TTM model re-initializes from disk on every tool call, loading it once at server startup and compiling the computation graph eliminates repeated cold-start overhead and fuses 19,032 CUDA kernel launches into fewer, faster operations.

2. Trainer Replacement with Direct Model Call

The HuggingFace Trainer wraps inference in a DataLoader loop adding 458ms of unnecessary per-batch memcpy and tensor allocation overhead, a direct batched model call processes all samples in one shot.

3. Mixed-Precision Inference (bf16)

TTM runs in float32 by default, wrapping the forward pass in torch.autocast(dtype=torch.bfloat16) halves activation memory and enables Tensor Core acceleration on the T4 GPU.

Profiling Methodology

Tools

- PyTorch Profiler (torch.profiler) - kernel-level CPU/CUDA timing, 19,032 kernel launches identified per forward pass
- Weights & Biases - run-level metric tracking across all optimization configurations
- Custom TSFMProfiler - per-stage wall-clock timing (data loading, preprocessing, model loading, ttm_forward, postprocessing)

Metrics captured

- Per-stage latency: data loading, preprocessing, model loading, ttm_forward, postprocessing
- Total workflow completion latency (end-to-end)
- Peak GPU memory (VRAM) per stage in MB
- Model loading overhead per tool call
- n_sensors scaling (1, 2, 5, 9 sensors)
- Speedup ratio across optimization configurations (baseline, GPU, bf16)

Experimental Setup

Setting	Baseline	Optimized
Precision	[FP32]	[FP32 + BF16]
Device	CPU	NVIDIA T4 GPU
Model loading	Per tool call	Pre-loaded at startup
Trainer	HuggingFace Trainer	Direct model(**inputs) call
Hardware	GCP n1-standard-4, T4 16GB	GCP n1-standard-4, T4 16GB
Software stack	PyTorch 2.6.0+cu124, granite-tsfm 0.3.6	[+torch.compile]

Pipeline: data loading, pre processing, model loading, ttm_forward, post processing

Results — Headline Numbers

3.3x

Faster inference

40,161 ms → 16,000 ms (model loading)
torch.compile + kernel fusion

68%

Reduction in in total workflow latency

43,021 ms → 13,798 ms

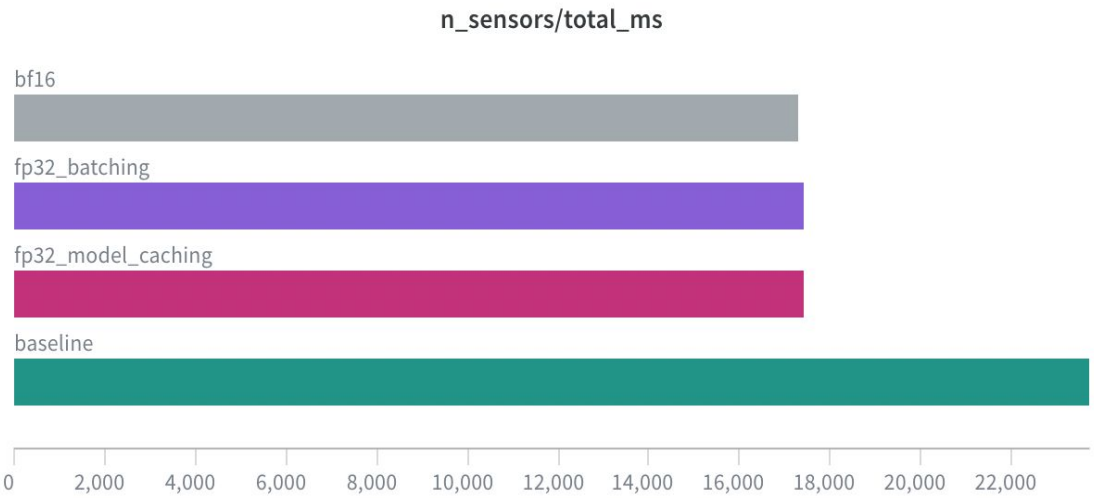
9x

9x faster model loading overhead per tool call

233 ms → 26 ms

All measurements averaged over 3 runs for 1,2,5,9 sensors.

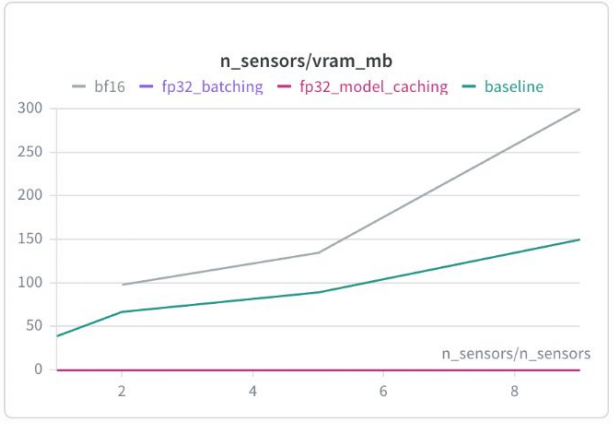
Results — Before vs. After

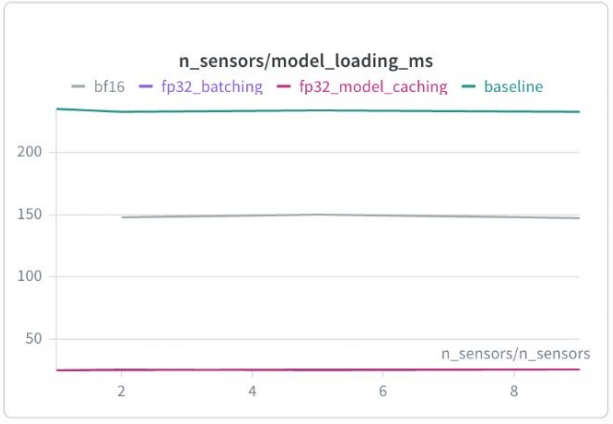


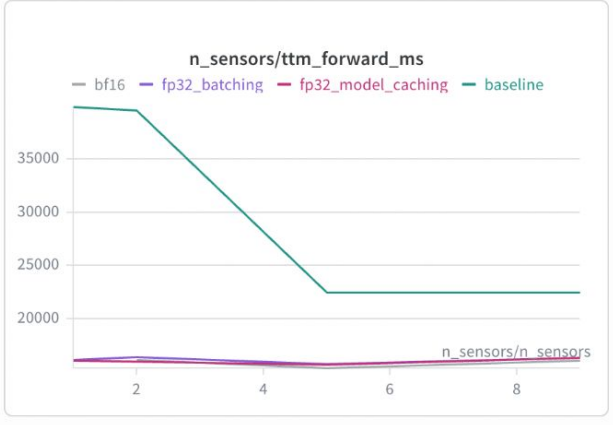
Takeaway

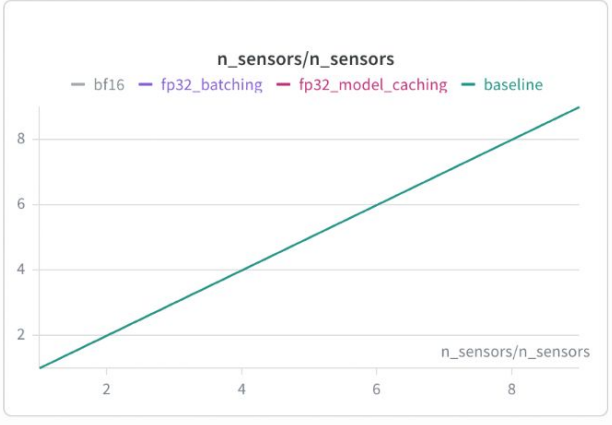
- Total workflow latency reduced from 40,161 ms to 16,000 ms - **3.3x speedup** from torch.compile + model caching
- All three optimized configs (bf16, fp32_batching, fp32_model_caching) are identical at ~17,000ms - confirming **model caching is the dominant optimization**
- Adding GPU placement on top brings total latency to **13,798ms - 3.3x vs the true baseline of 43,021ms**
- bf16 shows no benefit over fp32, TTM's MLP architecture does not leverage Tensor Core acceleration

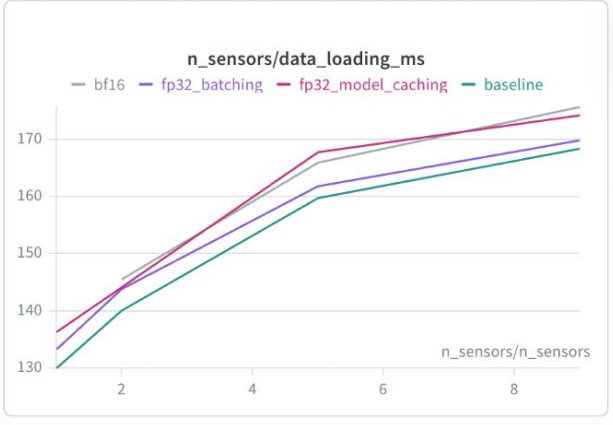
Results - comparison of optimization across metrics











Profiler Evidence

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self CUDA %	CUDA total	CUDA time avg	# of Calls
_compile.compile_inner (dynamo_timed)	0.00%	0.000us	0.00%	0.000us	0.000us	1.571s	20894.51%	1.571s	1.571s	1
create_aot_dispatcher_function (dynamo_timed)	0.00%	0.000us	0.00%	0.000us	0.000us	668.954ms	8899.62%	668.954ms	668.954ms	1
CUDAGraphTreeManager.run_eager (dynamo_timed)	0.00%	0.000us	0.00%	0.000us	0.000us	209.817ms	2791.35%	209.817ms	104.908ms	2
_recursive_joint_graph_passes (dynamo_timed)	0.00%	0.000us	0.00%	0.000us	0.000us	205.628ms	2735.63%	205.628ms	205.628ms	1
compile_fx_inner (dynamo_timed)	0.00%	0.000us	0.00%	0.000us	0.000us	132.674ms	1765.07%	132.674ms	132.674ms	1
Unrecognized	1.20%	108.170ms	1.20%	108.170ms	236.697us	8.328ms	110.80%	8.328ms	18.224us	457
aten::copy_	0.02%	1.983ms	0.07%	6.403ms	57.684us	1.537ms	20.44%	1.537ms	13.844us	111
Memcpy HtoD (Pageable -> Device)	0.00%	0.000us	0.00%	0.000us	0.000us	1.476ms	19.63%	1.476ms	43.403us	34
aten::mm	1.30%	117.274ms	2.38%	214.128ms	2.933ms	1.184ms	15.75%	8.973ms	122.916us	73
void cutlass::Kernel2<cutlass_80_wmma_tensorop_bf16_...	0.00%	0.000us	0.00%	0.000us	0.000us	1.050ms	13.96%	1.050ms	52.483us	20

Self CPU time total: 9.014s
Self CUDA time total: 7.517ms

--- CPU ---

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU time avg	Self CUDA	Self CUDA %	CUDA total	CUDA time avg	# of Calls
_compile.compile_inner (dynamo_timed)	31.07%	2.801s	93.61%	8.438s	2.813s	0.000us	0.00%	49.409us	16.470us	3
compile_fx_inner (dynamo_timed)	12.44%	1.121s	23.67%	2.134s	1.067s	0.000us	0.00%	2.720us	1.360us	2
async_compile.wait (dynamo_timed)	4.71%	424.897ms	4.74%	427.548ms	213.774ms	0.000us	0.00%	0.000us	0.000us	2
PyCodeCache.load_by_key_path (dynamo_timed)	4.17%	375.667ms	8.91%	803.215ms	401.608ms	0.000us	0.00%	0.000us	0.000us	2
create_aot_dispatcher_function (dynamo_timed)	3.83%	345.420ms	51.02%	4.599s	2.299s	0.000us	0.00%	45.442us	22.721us	2
aten::_to_copy	3.71%	334.195ms	7.95%	716.919ms	793.052us	0.000us	0.00%	1.502ms	1.661us	904
_recursive_joint_graph_passes (dynamo_timed)	3.58%	322.568ms	4.86%	438.416ms	219.208ms	0.000us	0.00%	37.505us	18.752us	2
aten::view	3.40%	306.297ms	4.12%	371.428ms	518.031us	0.000us	0.00%	0.000us	0.000us	717
Torch-Compiled Region: 0/0	3.28%	295.591ms	8.15%	734.589ms	734.589ms	0.000us	0.00%	14.303ms	14.303ms	1
prims::convert_element_type	2.93%	264.179ms	5.09%	459.170ms	581.964us	0.000us	0.00%	0.000us	0.000us	789

Self CPU time total: 9.014s
Self CUDA time total: 7.517ms

Profiler Interpretation

Key findings

- The trace is dominated by torch.compile warmup :
  _compile.compile_inner: **~8.4s CPU total**
- CUDA execution is active, but not the main bottleneck:
  Self CUDA time total: **~7.5ms**
- GPU kernels are present:
  aten::mm and CUTLASS BF16 Tensor Core kernels appear in the trace.
- Data transfer is small:
  Memcpy HtoD: **~1.5ms total**
- BF16 is active, but not enough by itself:
  Copy and dtype conversion overhead reduces the benefit.

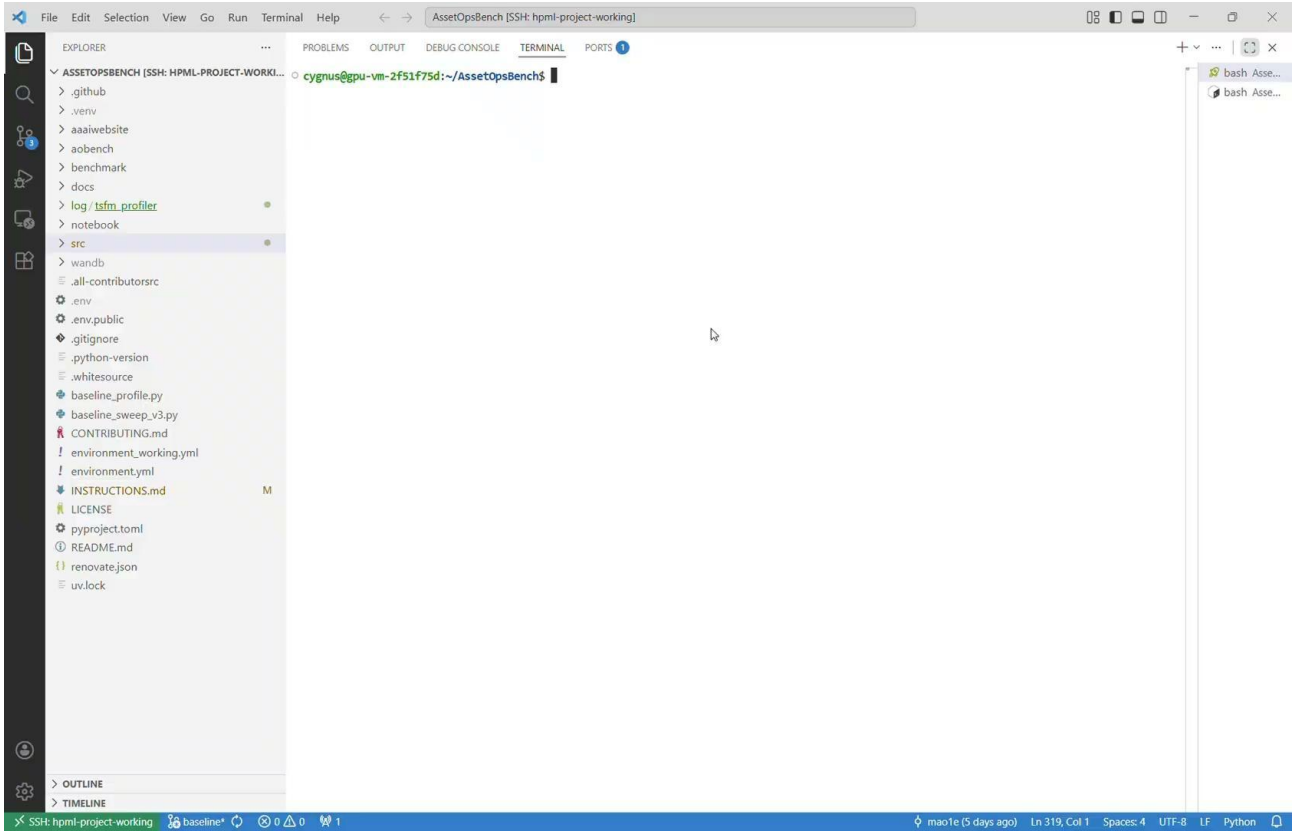
Main interpretation

The first compiled call is CPU/compiler dominated. The optimization only pays off when the model is compiled once, cached at server startup, and reused for later MCP tool calls.

Final takeaway

For TinyTimeMixer, the biggest win came from **model caching + direct compiled inference**, not from BF16 alone.

Demo / Visualization



Ablation study

Optimization Stack	ttm_forward	Total	Speedup
Baseline CPU	~38,000ms	~41,000ms	1x
+ Model caching	~16,000ms	~17,000ms	2.4x
+ bf16	~16,000ms	~17,000ms	same
+ using GPU	~12,293ms	~13,798ms	3.3x

Lessons Learned

Time budget: 1 minute. What surprised you? What didn't work? Be honest.

What worked

torch.compile: 2.5x speedup on ttm_forward. Kernel fusion eliminated 19,032 CUDA launches.

Direct model calls: Replacing HF Trainer eliminated DataLoader and memcpy overhead per inference.

Model pre-loading: Reduced loading time from ~230ms to ~25ms (9x faster per tool call).

GPU placement: Unlocked additional 1.3x speedup; total 3.3x vs baseline.

What didn't work

bf16 precision: Performance dropped (15,272ms vs 12,293ms fp32). TTM's MLP structure didn't benefit Tensor Cores.

Batching: No impact. Orchestrator LLM passes sensors in a single call, leaving no serial behavior to collapse.

GPU Idle Assumption: TTM is CPU-optimized by IBM; GPU gains are modest unlike attention-based models.

Next Steps

Time budget: 30 seconds. Be specific — vague "more experiments" is not next steps.

FastAPI
Concurrent tool calls

INT8 Quantization via bitsandbytes
Reduce ttm_forward memory bandwidth

Full AssetOpsBench Evaluation
Extend to scenarios 216-223 using official chiller9 dataset once access is obtained from IBM.

Teamwork & Contributions

Time budget: 30 seconds. Who did what. Required slide — graded individually.

Member	Primary contributions	Tools / artifacts
Alisha	Baseline sweep, Wandb logging, mixed precision	baseline_profile.py, baseline_sweep.py
Thomas	Report writing, results analysis, figures, resolving dependency/integration conflicts	
Jonathan	Model caching	forecasting.py, model_cache.py
Sanjaii	Initial baseline setup, building configurations, testing out the plan-execute pipeline, brainstorming ideas for optimization.	

Collaboration: regular [twice-weekly] meetings via [n person], shared [Whatsapp] channel

Thank you

Questions?

GitHub: https://github.com/alishavinod/AssetOpsBench/tree/baseline
Tracking: https://wandb.ai/av3311-columbia-university/hpml-tsfm-optimization/workspace?nw=nwuserav3311